

Vector Stores and Retrievers

Concept	What it Means	Why It's Useful
Documents	Small chunks of text + optional metadata	Organizes data for retrieval
Embeddings	Number representation of text (vectors)	Helps computers understand similarity in meaning
Vector Store	A database of vectors	Used to find similar text quickly
Retriever	A tool that fetches similar documents from a vector store	Used in RAG (Retrieval-Augmented Generation)
RAG Chain	An LLM + Retriever working together	Gives LLM access to external knowledge

What is a Document?

- A **Document** is a core abstraction representing a **chunk of text + metadata**.
- Used to store:
 - **page_content**: The actual text (e.g., "Dogs are great companions...")
 - **metadata**: Additional info (e.g., **{"source": "mammal-pets-doc"}**).

```
from langchain_core.documents import Document
```

```
documents = [  
    Document(  
        page_content="Dogs are great companions...",  
        metadata={"source": "mammal-pets-doc"},  
    ),  
    ...  
]
```

```

from langchain_core.documents import Document

documents = [
    Document(
        page_content="Dogs are great companions, known for their loyalty and f
riendliness.",
        metadata={"source": "mammal-pets-doc"},
    ),
    Document(
        page_content="Cats are independent pets that often enjoy their own spa
ce.",
        metadata={"source": "mammal-pets-doc"},
    ),
    Document(
        page_content="Goldfish are popular pets for beginners, requiring relative
ly simple care.",
        metadata={"source": "fish-pets-doc"},
    ),
    Document(
        page_content="Parrots are intelligent birds capable of mimicking human
speech.",
        metadata={"source": "bird-pets-doc"},
    ),
    Document(
        page_content="Rabbits are social animals that need plenty of space to h
op around.",
        metadata={"source": "mammal-pets-doc"},
    ),
]

```

- 🙌 This is a list of documents

🔍 Components:

- `page_content`: actual text content

- `metadata`: optional key-value tags like source name, author, etc.

Why?

These documents are often **split into chunks** and stored so that we can later find them when we need relevant information.

Set env components

```
import os
from dotenv import load_dotenv
load_dotenv()
from langchain_groq import ChatGroq
groq_api_key=os.getenv("GROQ_API_KEY")

os.environ["HF_TOKEN"]=os.getenv("HF_TOKEN")
```

Load LLM:

```
llm=ChatGroq(groq_api_key=groq_api_key,model="Llama3-8b-8192")
llm
```

Embeddings (HuggingFace)

To find similar text, we convert it into numbers (vectors) using **embeddings**.

text → vectors

```
from langchain_huggingface import HuggingFaceEmbeddings
```

```
embeddings=HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
```

**!! This will download the model @
C:\Users\Jeevan\.cache\huggingface\hub
Size ~88 MB**

all-MiniLM-L6-v2

This is a [sentence-transformers](#) model: It maps sentences & paragraphs to a 384 dimensional dense vector space and can be used for tasks like clustering or semantic search.

<https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>

💡 Internally:

- The embedding model turns sentences into lists of floats like `[0.123, -0.33, 0.45, ...]`
- These numbers **preserve meaning**. So "cat" and "kitten" have similar embeddings.

Vector Store (Chroma)

```
langchain_chroma
```

- This is a **special database** that stores document embeddings and allows fast searching by meaning.

```
from langchain_chroma import Chroma
```

```
vectorstore = Chroma.from_documents(documents, embedding=embeddings)  
vectorstore
```

```
<langchain_chroma.vectorstores.Chroma at 0x248680e6020>
```

Similarity Search in the vector store:

```
vectorstore.similarity_search("cat")
```

```
[Document(page_content='Cats are independent pets that often enjoy their own space.', metadata={'source': 'mammal-pets-doc'}),  
Document(page_content='Dogs are great companions, known for their loyalty and friendliness.', metadata={'source': 'mammal-pets-doc'}),  
Document(page_content='Rabbits are social animals that need plenty of space to hop around.', metadata={'source': 'mammal-pets-doc'}),  
Document(page_content='Parrots are intelligent birds capable of mimicking human speech.', metadata={'source': 'bird-pets-doc'})]
```

Async query

- An **"async query"** (short for **asynchronous query**) is a way of asking for data **without making your program wait** while the request is being processed.

```
await vectorstore.asimilarity_search("cat")
```

```
[Document(page_content='Cats are independent pets that often enjoy their own space.', metadata={'source': 'mammal-pets-doc'}),  
Document(page_content='Dogs are great companions, known for their loyalty and friendliness.', metadata={'source': 'mammal-pets-doc'}),  
Document(page_content='Rabbits are social animals that need plenty of space to hop around.', metadata={'source': 'mammal-pets-doc'}),  
Document(page_content='Parrots are intelligent birds capable of mimicking human speech.', metadata={'source': 'bird-pets-doc'})]
```

Similarity search with score:

```
vectorstore.similarity_search_with_score("cat")
```

```
[(Document(id='999c9869-30ab-47f8-ba34-8aaba9586063', metadata={'source': 'mammal-pets-doc'}, page_content='Cats are indep  
0.9351057410240173),  
(Document(id='07cd3d8f-2c42-49c9-a50f-a6ad828929c3', metadata={'source': 'mammal-pets-doc'}, page_content='Dogs are great  
1.5740900039672852),  
(Document(id='2ff509dd-b3c9-4ae2-84e3-2c68828a8054', metadata={'source': 'mammal-pets-doc'}, page_content='Rabbits are so  
1.5956902503967285),  
(Document(id='0a96b112-50ea-4fd4-ad2f-e88392497b8f', metadata={'source': 'bird-pets-doc'}, page_content='Parrots are inte  
1.6657923460006714)]
```

- 👉 0.93 is the nearest.

Retriever

- Retriever is a **runnable** component that fetches relevant documents for an LLM.

The vectorstore isn't directly compatible with LangChain Expression Language (LCEL), so we convert it into a Retriever.

Create a Retriever:

(Recommended) 

```
retriever = vectorstore.as_retriever(
    search_type="similarity", #default
    search_kwargs={"k": 1} # return top 1 most similar doc
)

retriever.batch(["cat","dog"])
```

```
[[Document(page_content='Cats are independent pets that often enjoy their own space.', metadata={'source': 'Cats'}),
 Document(page_content='Dogs are great companions, known for their loyalty and friendliness.', metadata={'source': 'Dogs'})]]
```

 Available **search_type** values (LangChain supports):

search_type	What it does
"similarity"	Returns documents closest to your query in vector space (default)
"mmr"	Uses Maximal Marginal Relevance : balances relevance + diversity

search_type	What it does
"similarity_score_threshold"	Returns documents above a score threshold only

1. "similarity" (default)

- Returns top **k** documents most similar to the query.
- Simple cosine similarity or distance-based.

2. "mmr" (Maximal Marginal Relevance)

- Tries to return documents that are:
 -  Relevant to your query
 -  Not too similar to each other (diverse)
- Good for long answers where you want a variety of context.

```
vectorstore.as_retriever(
    search_type="mmr",
    search_kwargs={"k": 5, "fetch_k": 10, "lambda_mult": 0.5}
)
```

Parameters:

- **k**: how many docs to return
- **fetch_k**: how many to initially retrieve before filtering
- **lambda_mult**: how much to prioritize relevance vs diversity
(0 = pure diversity, 1 = pure relevance)

3. "similarity_score_threshold"

- Filters out documents that aren't **similar enough**
- Good when you want to **ignore weak matches**

```
vectorstore.as_retriever(
    search_type="similarity_score_threshold",
```

```
search_kwargs={"score_threshold": 0.7, "k": 5}
)
```

Manual Method: (Not Recommended ❌)

```
from langchain_core.runnables import RunnableLambda

retriever = RunnableLambda(vectorstore.similarity_search).bind(k=1)
retriever.batch(["cat", "dog"]) # Batch search
```

```
[[Document(page_content='Cats are independent pets that often enjoy their own space.', metadata={'source': 'cat'}),
 Document(page_content='Dogs are great companions, known for their loyalty and friendliness.', metadata={'source': 'dog'})]]
```

✅ Standard LangChain Retriever Methods

Method	Description
<code>.invoke(input)</code>	Get documents for one query string
<code>.batch([queries])</code>	Get documents for a list of queries
<code>.ainvoke(input)</code>	Async version of <code>.invoke()</code>
<code>.abatch([queries])</code>	Async version of <code>.batch()</code>

RAG Chain (Retriever + LLM)

RAG = Retrieval-Augmented Generation

What is RAG?

- Combines **retrieval** (fetching docs) + **generation** (LLM reasoning).

Building a RAG Chain:

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
```



```

# Prompt template
prompt = ChatPromptTemplate.from_messages([
    ("human", """
    Answer this question using the provided context only.

    {question}

    Context:
    {context}
    """)
])

# RAG chain
rag_chain = {
    "context": retriever,
    "question": RunnablePassthrough()
} | prompt | llm # llm = ChatGroq/Llama3

# Invoke
response = rag_chain.invoke("tell me about dogs")
print(response.content)

```

```

According to the provided context, dogs are:

* Great companions
* Known for their loyalty
* Known for their friendliness

```

- **RunnablePassthrough()** preserves the raw input (question) without modification, while allowing other components (like the retriever) to dynamically add new data (context) to the chain.



`RunnablePassthrough()` : We will get this parameter in the `invoke` method.



`rag_chain. invoke ("tell me about dogs")`

`rag_chain=`

`{"context":retriever,"question":RunnablePassthrough()}|prompt|llm :`

- This is the core of your RAG system – it's a **chain** of operations linked together by the `|` (pipe) symbol.
- The pipe means "**take the output from the left side and feed it as input to the right side.**"
- `"context": retriever` : This tells LangChain, "When a question comes in, first use the `retriever` to find relevant information (the '**context**') related to that question."
- `"question": RunnablePassthrough()` : This simply takes the *original question* you provided to the chain (`"tell me about dogs"`) and passes it along directly.